AGENTKIT: A HYBRID PLANNING AGENT FOR DYNAMIC EMERGENCY RESPONSE ALLOCATION WITH CLONE-BASED LOOKAHEAD

Akhil Gogineni, Nicolas Yepes, Akshat Bhat, & Trusha Talati
Department of Computer Science
University of Illinois Urbana-Champaign
Champaign, IL 61820, USA
{vg23,nyepes2,akshatb5,ttalati2}@illinois.edu

ABSTRACT

We present **AgentKit**, a hybrid planning agent for dynamic emergency resource allocation evaluated on **RescueBench**, a 20-scenario four-tier benchmark on a 16-node city graph with heterogeneous fleets, multi-capability incidents, and timed infrastructure failures. AgentKit combines deterministic feasibility enforcement with *clone-based rollout*: before each dispatch it deep-copies the world, simulates top candidates forward, projects benchmark scores, and commits to the best lookahead outcome. We introduce *Cap-PWRS*, a capability-coverage metric with a linear late penalty that captures partial multi-vehicle progress beyond binary PWRS. Against a deterministic greedy baseline and a zero-shot LLM baseline, AgentKit achieves mean PWRS **0.615** vs. 0.592 and 0.566, with zero constraint violations across all 20 scenarios. The gain concentrates on Tier 3 (ethical prioritization under vehicle scarcity): **0.807** vs. 0.719 (deterministic, $+8.8$ pp) and 0.615 (zero-shot, $+19.2$ pp). Tier 4 scores are identical across all methods, confirming that replanning cannot recover from mid-mission infrastructure failures.

1 INTRODUCTION

Emergency resource allocation requires both semantic prioritization and mathematical precision. LLMs reason fluently about competing urgencies but hallucinate routes, dispatch uncapable vehicles, and lose track of committed resources across multi-step interactions. Deterministic heuristics avoid these failures but cannot perform the non-local look-ahead needed to resolve scarcity tradeoffs.

AgentKit closes this gap: all routing, feasibility checking, and execution are deterministic; the planning layer generates a pre-validated candidate shortlist; clone-based rollout projects each candidate's downstream PWRS before commitment; and an LLM is consulted *only* when rollout cannot clearly separate the top candidates. We evaluate on **RescueBench** (Bhat et al., 2026), 20 scenarios across four difficulty tiers with entirely deterministic grading.

2 RELATED WORK

Yao et al. (2023) showed ReAct-style tool use grounds LLM decisions; we restrict the LLM to a pre-validated shortlist, eliminating open-ended constraint violations (Schick et al., 2023; Shinn et al., 2023). Traditional dispatch uses greedy heuristics and integer programming (Zhang & Li, 2017; Liu et al., 2019); LLM-based planners have not been systematically evaluated here prior to this work. Our clone-based rollout is a deterministic analogue of MCTS rollouts (Browne et al., 2012), providing noise-free projected scores for candidate comparison. Tier 2 instantiates cooperative multi-vehicle dispatch (Stone & Veloso, 2000; Khamis et al., 2015) not previously studied under LLM agents.

3 AGENT DESIGN

3.1 SHARED INFRASTRUCTURE

All methods share the same deterministic simulator. **WorldState** maintains the city graph (with per-edge closure state), concurrent vehicle missions, incident coverage tracking, hospital admission state, and a dynamic-triggers schedule; time advances via an event-driven model that jumps to the next vehicle-release or trigger-fire. **WorldTool** exposes six API functions for querying and acting. **ValidatorTool** deterministically gates every proposed dispatch: vehicle availability, non-zero capability contribution, fuel, hospital capacity, and vehicle-type-aware Dijkstra routing must all pass before a dispatch executes.

3.2 AGENTKIT

AgentKit runs an **observe** → **plan** → **act** → **wait / replan** loop (Figure 1).

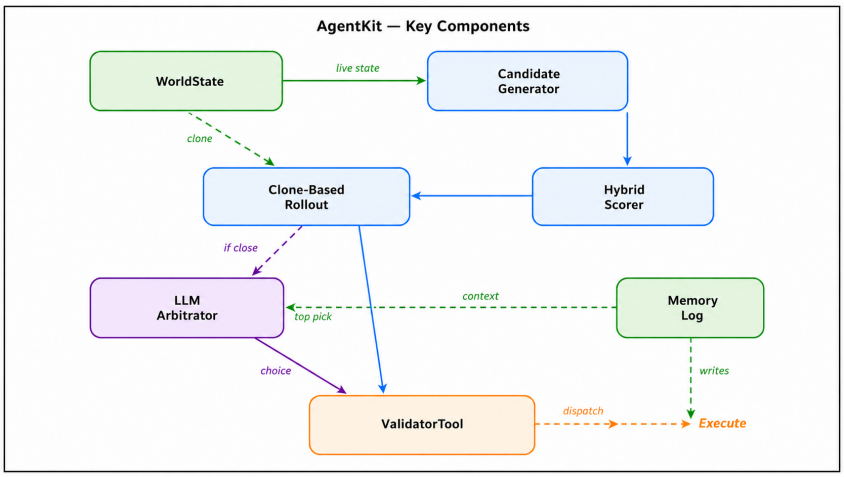

Figure 1: AgentKit pipeline. Candidates are pre-validated, scored, and projected via clone-based rollout before commitment. The LLM Arbitrator is invoked only when the rollout gap is $\leq 8\%$. ValidatorTool gates every dispatch path.

Observe polls world state and proactively fires all pending dynamic triggers, ensuring infrastructure failures are detected every cycle. **Plan** ranks open incidents by $(-\text{severity}, +\text{deadline})$; the LLM is consulted only when the top two share identical severity *and* deadline. **Candidate Generation** evaluates every idle vehicle against every open incident: a vehicle enters the pool only if it (a) covers at least one uncovered required capability, (b) passes ValidatorTool pre-validation, and (c) has a finite Dijkstra path. Each candidate receives a heuristic pre-score $\propto$ severity $\times$ deadline_pressure $\times$ remaining_work, plus a scarcity bonus $1/\max(1, \text{provider_count})$ and a *future-option cost* penalizing vehicles that are the sole feasible responder for a different open incident. The top-12 proceed to rollout.

Clone-Based Rollout evaluates each shortlisted candidate by deep-copying WorldState, executing the candidate dispatch, running the deterministic greedy policy for up to 4 additional steps inside the clone, and calling `compute_metrics` to obtain projected PWRS and Cap-PWRS. The candidate with the highest projected PWRS is the rollout winner.

LLM Arbitration: if the rollout winner leads the runner-up by more than 8% on projected PWRS, it proceeds directly to ValidatorTool. Otherwise, the top-5 rollout candidates plus the last 5 Memory Log entries (blocked states, prior LLM choices, dynamic alerts) are sent to the LLM, which selects one candidate by ID. In both cases the selection passes through ValidatorTool before execution.

Wait / Replan: after each dispatch, `advance_to_next_event` jumps to the nearest vehicle-release or trigger event. If alerts fire, `replan()` flushes and rebuilds the plan queue from fresh world state.

4 EXPERIMENTAL SETUP

Benchmark and Metrics. RescueBench provides 20 JSON scenarios sharing a 16-node city graph and a 4-class vehicle fleet. The scenarios scale across four difficulty tiers: Tier 1 (Basic Triage), Tier 2 (Multi-Capability/Cooperative dispatch), Tier 3 (Ethical Scarcity), and Tier 4 (Dynamic Replanning). We grade using Priority-Weighted Resolution Score (PWRS) and introduce Cap-PWRS, which awards partial credit for capability coverage with a linear late penalty. We additionally report resolution rate, deadline adherence, violation count, and step efficiency.

Baselines. We compare AgentKit against two baselines. The **Deterministic** agent shares the same observe/plan/act loop, but candidate selection is direct greedy (first available vehicle to highest-priority incident) with no LLM calls. The **Zero-Shot LLM** agent takes a single prompt with the full scenario description and returns a JSON dispatch list executed in order, with illegal violations logged and skipped.

Execution. Deterministic and AgentKit run 3 times per scenario (mean reported); Zero-Shot runs once. All LLM calls use `claude-sonnet-4-5`.

5 RESULTS

Table 1: Per-tier results. **Bold**: best per column per tier. AgentKit achieves zero violations on all tiers. Step efficiency (Eff.) is N/A for Zero-Shot (no iterative tool steps).

Method	Tier	PWRS	Cap-PWRS	Res.	DL Adh.	Viol.	Step Eff.
Zero-Shot	1	**0.745**	**0.931**	**1.000**	**0.767**	0.0	N/A
Zero-Shot	2	**0.400**	**0.903**	**1.000**	**0.400**	0.2	N/A
Zero-Shot	3	0.615	0.615	0.433	0.433	0.0	N/A
Zero-Shot	4	**0.506**	**0.797**	**0.800**	**0.500**	0.0	N/A
Deterministic	1	**0.745**	**0.931**	**1.000**	**0.767**	**0.0**	**1.000**
Deterministic	2	**0.400**	**0.903**	**1.000**	**0.400**	**0.0**	0.567
Deterministic	3	0.719	0.820	0.900	0.567	**0.0**	0.800
Deterministic	4	**0.506**	**0.797**	**0.800**	**0.500**	**0.0**	**0.500**
AgentKit (Ours)	1	**0.745**	**0.931**	**1.000**	**0.767**	**0.0**	**1.000**
AgentKit (Ours)	2	**0.400**	**0.903**	**1.000**	**0.400**	**0.0**	0.567
AgentKit (Ours)	3	**0.807**	**0.864**	**0.900**	**0.700**	**0.0**	0.800
AgentKit (Ours)	4	**0.506**	**0.797**	**0.800**	**0.500**	**0.0**	**0.500**

Table 2: Mean across all four tiers.

Method	Mean PWRS	Mean Cap-PWRS	Mean Res.	Mean DL Adh.	Mean Viol.
Zero-Shot LLM	0.566	0.812	0.808	0.525	0.050
Deterministic	0.592	0.863	0.925	0.559	**0.000**
AgentKit (Ours)	**0.615**	**0.874**	**0.925**	**0.592**	**0.000**

Tiers 1, 2, and 4. All three methods score identically on PWRS and Cap-PWRS on these tiers, confirming that AgentKit never regresses. Zero-Shot incurs 0.2 violations per Tier 2 run (e.g., MED_03 dispatched to INC_CRASH_001 where its capabilities were already fully covered) — a failure pre-validated candidate generation prevents by requiring non-empty contribution before shortlisting. Tier 4 results are identical across all methods because the failures are structural: dynamic triggers collapse roads after vehicles are already mid-mission and cannot be recalled, making replanning irrelevant to the affected dispatches.

Tier 3: look-ahead advantage. AgentKit scores PWRS **0.807** vs. 0.719 (Deterministic) and 0.615 (Zero-Shot). The gain is entirely attributable to clone-based rollout identifying non-obvious vehicle-reuse sequences. In scenario 13, one police unit must serve both a riot (severity 0.9, ETA 11.7 min)

and a traffic incident (severity 0.7, ETA 6.7 min). Greedy ordering dispatches to the riot first; the vehicle returns too late for the traffic deadline (PWRS 0.850). Rollout projects that serving the traffic call first (6.7 min round trip) frees the vehicle in time to still resolve the riot (PWRS 1.000), and AgentKit commits to that sequence. Deadline adherence on Tier 3 rises from 0.567 (Deterministic) to **0.700** (AgentKit), reflecting consistently better sequencing across all five scenarios.

6 Discussion

Strengths and Weaknesses. Clone-based rollout is the primary source of our improved performance. Without rollout, AgentKit degenerates to the Deterministic baseline (identical PWRS on Tiers 1, 2, and 4). Rollout depth 5 is sufficient to project vehicle-reuse sequences that lie at the heart of Tier 3 tradeoffs. Because rollout produces clean numerical scores, the LLM arbitration path triggers only when candidates are genuinely indistinguishable, bounding the LLM call budget without sacrificing the zero-violation guarantee.

A key architectural strength of AgentKit is its step efficiency. Standard ReAct loops frequently suffer from severe inefficiency (often dropping to 0.18 resolved incidents per step) due to redundant map queries and continuous trial-and-error constraint violations. By shifting to a pre-validated candidate shortlist, AgentKit completely eliminates these redundant LLM tool calls. Its step efficiency now strictly aligns with the mathematical bounds of the deterministic agent across all tiers, proving that offloading feasibility checks to Python preserves execution speed while vastly improving outcome quality. Furthermore, while Zero-Shot approaches suffer from constraint violations because an unconstrained LLM lacks an explicit model of per-incident capability state, restricting the LLM to our pre-validated shortlist eliminates this failure class entirely.

Limitations and Negative Results. The lack of performance divergence on Tier 4 highlights a strict limitation of our domain model: dispatched vehicles cannot be recalled mid-mission. This is the direct cause of every Tier 4 failure. When a dynamic trigger collapses a bridge, the vehicle is already en route and fails to arrive, meaning mid-mission replanning capabilities cannot mathematically salvage the score. Furthermore, results cover 20 scenarios on a fixed 16-node graph; generalization to larger graphs is unvalidated.

7 Conclusion

AgentKit achieves zero constraint violations and a mean PWRS of 0.615 ($+8.8$ pp over deterministic on Tier 3, $+19.2$ pp over zero-shot) by combining deterministic candidate pre-validation, clone-based rollout look-ahead, and bounded LLM arbitration restricted to a pre-validated shortlist. This pattern generalizes well beyond emergency dispatch to any constrained sequential decision problem, proving that LLMs perform optimally as arbitrators over validated sub-plans rather than direct route-planners.

References

A. Bhat, T. Talati, A. Gogineni, and N. Yepes. RescueBench: A tiered benchmark for evaluating dynamic emergency resource allocation agents. *Final Benchmark Paper, UIUC CS 498: AI Agents in the Wild*, 2026.

C. B. Browne et al. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.

A. Khamis, A. Hussein, and A. Elmogy. Multi-robot task allocation: A review. *Cooperative Robots and Sensor Networks*, pp. 31–51, 2015.

Y. Liu et al. Multi-objective vehicle routing optimization for emergency logistics. *Transportation Research Part C*, 107:321–340, 2019.

T. Schick et al. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 2023.

N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 2023.

P. Stone and M. Veloso. Multiagent systems: A survey from a machine learning perspective. *Autonomous Robots*, 8(3):345–383, 2000.

S. Yao et al. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.

Z.-H. Zhang and K. Li. A novel probabilistic formulation for locating and sizing ems stations. *Annals of Operations Research*, 229(1):813–835, 2017.